INSTITUTO POLITECNICO NACIONAL

UNIDAD PROFESIONAL

INTERDICIPLINARIA DE INGENERIA Y CIENCIAS SOCIALES

ADMJNISTRATIVAS

Materia: Ingería de pruebas

Equipo:

Rodríguez Mendiola Valentina

Martínez Lagunas Andrik Jeovany

Mejía Ramírez Luis Alejandro

Jimenez Rogel Sergio

Fecha de Elaboración: 19/12/2025

BookSync – Biblioteca Interactiva y Personalizable

Descripción del proyecto

BookSync es una aplicación móvil desarrollada en Kotlin con Jetpack Compose que permite gestionar una biblioteca personal de manera eficiente y personalizada. La app permite:

- Agregar, editar y eliminar libros.

- Sincronizar datos entre una base local (SQLite) y una base remota (MySQL) mediante conexión local.

- Filtrar libros por título, autor o género.

- Marcar libros como favoritos.

- Personalizar los colores de la interfaz según las preferencias del usuario.

- Acceder a una pantalla de ayuda.

- Utilizar un menú desplegable para facilitar la navegación entre secciones clave.

Todo esto con una interfaz moderna, fluida y adaptable.

¿Cómo funciona?

BookSync combina almacenamiento local y remoto:

- Los libros se guardan tanto en una base de datos local (SQLite) como en una base remota (MySQL).

- Se implementa sincronización bidireccional, lo que permite que los datos locales se suban al servidor remoto y viceversa.

- Se utiliza una API escrita en *PHP* que expone endpoints para operaciones CRUD (crear, leer, actualizar y eliminar), facilitando la comunicación entre la app y el servidor.

- Al iniciar la app o realizar ciertas acciones, la base de datos local se actualiza con los datos más recientes del servidor.

- La sincronización también detecta registros nuevos o modificados comparando las fechas de actualización.

Tecnologías y herramientas utilizadas

Android Studio – Entorno de desarrollo principal.

Kotlin – Lenguaje de programación.

- Jetpack Compose – Toolkit para la construcción de la interfaz.

- SQLite – Base de datos local en el dispositivo.

- USB Web Server – Servidor local que incluye:

  - Apache para servir archivos PHP.

  - MySQL como base de datos remota.

  - phpMyAdmin para gestión visual de la base de datos.

- PHP – Backend para manejar solicitudes HTTP de la app.

- HTTP (con OkHttp + Retrofit) – Comunicación entre app y servidor.

Estructura general de la aplicación

Pantalla principal: lista de todos los libros, obtenidos desde SQLite.

- Pantalla de agregar/editar: formulario completo para capturar o modificar datos de un libro.

- Menú desplegable: navegación entre:

  - Agregar nuevo libro

  - Filtrar libros

  - Favoritos

  - Personalización

  - Ayuda

- Pantalla de filtros: búsqueda por título, autor y género.

- Pantalla de favoritos: muestra libros marcados como favoritos.

- Pantalla de personalización: cambio de colores de fondo, texto y botones.

- Pantalla de ayuda: información útil sobre el uso de la app.

Puntos técnicos destacados del código

- Se utiliza SQLiteOpenHelper para gestionar la base de datos local de libros.

- La función sincronizar() realiza una sincronización desde la base remota (MySQL vía API REST) hacia la local, con lógica para evitar sobrescribir cambios no sincronizados o marcados como eliminados.

- La función subirCambiosPendientes() recorre los libros con cambios pendientes (pendiente_sync = 1) y los sube al servidor con llamadas HTTP (usando Retrofit o OkHttp).

- Se implementa un sistema de marcado lógico de eliminaciones (eliminado = 1) para evitar perder datos localmente antes de sincronizar.

- Las inserciones locales de libros usan insertarLibroOffline(), marcando como pendiente_sync = 1 para su posterior subida al servidor.

- Se usa *corutinas (suspend) y Dispatchers.IO* para operaciones de red no bloqueantes.

- La validación contra duplicados se realiza en libroDuplicado() comparando título, autor y fecha de publicación.

- Las URLs relativas se ajustan con arreglarUrl() para garantizar enlaces completos a imágenes y archivos PDF.

Requisitos técnicos y consideraciones de implementación

Para poder ejecutar correctamente BookSync, se deben tener en cuenta los siguientes aspectos técnicos:

Conexión local y sincronización

- La app sincroniza libros entre SQLite (local) y MySQL (servidor) usando archivos *PHP* alojados en un servidor local (como USB Web Server).

- Asegúrate de tener el servidor local activo (Apache y MySQL encendidos) y que la IP local esté correctamente asignada en Constantes.BASE_URL.

Archivos PHP necesarios

- El backend está compuesto por varios scripts PHP (por ejemplo, guardar_libro.php, eliminar_libro.php, filtrar_libros.php, etc.).

- Estos deben estar dentro del directorio raíz de Apache (/root/usbwebserver/root o similar) y configurados correctamente para acceder a la base de datos.

Base de datos remota

- Debes crear una base de datos MySQL llamada mibase (o la que hayas usado) y una tabla libros con campos coincidentes con los del código.

- La tabla debe tener columnas como: Id, Titulo, Autor, Descripcion, UrlImagen, UrlPdf, Genero, Fecha_publicacion, Fecha_modificacion, eliminado.

Permisos y red

- Si pruebas desde un dispositivo físico Android, ambos (PC y dispositivo) deben estar en la *misma red WiFi* para que funcione la conexión local.

- Asegúrate de que la app tiene permisos de red y acceso a internet habilitados en el AndroidManifest.xml.

Personalización

- La app guarda preferencias de color usando SharedPreferences.

- Estas preferencias son persistentes y aplicadas automáticamente al reiniciar la app.

Manejo de sincronización

- Cambios realizados sin internet se guardan en SQLite y se marcan como pendiente_sync.

- Cuando se detecta conexión, los cambios pendientes se sincronizan automáticamente con el servidor.

- Se recomienda revisar las funciones subirCambiosPendientes() y sincronizar() en DBHelper.kt si se necesita adaptar o extender el comportamiento.

Librerías usadas

- Retrofit y OkHttp para comunicación HTTP.

- Gson para conversión de JSON.

- Coroutines para llamadas asíncronas no bloqueantes.